

Migrations Laravel - Conversion MySQL vers PostgreSQL

Vue d'ensemble

Ce package contient **89 migrations Laravel** générées automatiquement à partir de votre fichier SQL MySQL, entièrement compatibles avec **PostgreSQL** et respectant les bonnes pratiques Laravel.

Problème résolu

Votre erreur PostgreSQL était causée par :

- **Noms d'index trop longs** (limite PostgreSQL : 63 caractères)
- **Transactions bloquées** par des erreurs de syntaxe
- **Types de données incompatibles** entre MySQL et PostgreSQL

Solution fournie

Migrations Laravel complètes

- **89 migrations de tables** avec types PostgreSQL optimisés
- **1 migration de clés étrangères** avec 68 relations
- **1 seeder** avec toutes les données existantes (36 tables)

Structure du package

```
database/  
├── migrations/  
│   ├── 2025_08_14_053300_create_accuse_receptions_table.php  
│   ├── 2025_08_14_053301_create_adresses_table.php  
│   ├── ... (87 autres migrations de tables)  
│   └── 2025_08_14_063434_add_foreign_keys.php  
└── seeders/  
    └── DatabaseSeeder.php
```

Installation et utilisation

1. Copier les fichiers dans votre projet Laravel

Copier les migrations

```
cp database/migrations/* /path/to/your/laravel/database/migrations/
```

Copier le seeder

```
cp database/seeder/DatabaseSeeder.php /path/to/your/laravel/database/seeder/
```

2. Configuration PostgreSQL

Dans votre fichier `.env` :

```
DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5432
DB_DATABASE=your_database_name
DB_USERNAME=your_username
DB_PASSWORD=your_password
```

3. Exécuter les migrations

Créer la base de données (si nécessaire)

```
php artisan migrate:fresh
```

Ou exécuter les migrations

```
php artisan migrate
```

4. Insérer les données

Exécuter le seeder

```
php artisan db:seed
```

Fonctionnalités incluses

Types de données convertis

- `AUTO_INCREMENT` → `$table->id()`
- `BIGINT UNSIGNED` → `$table->unsignedBigInteger()`

- TINYINT(1) → \$table->boolean()
- VARCHAR(n) → \$table->string(n)
- TEXT → \$table->text()
- TIMESTAMP → \$table->timestamp()

Fonctionnalités Laravel

- **Timestamps automatiques** (created_at , updated_at)
- **Soft deletes** (deleted_at)
- **Clés primaires** auto-incrémentées
- **Index** optimisés
- **Clés étrangères** avec contraintes

Données préservées

- **36 tables** avec données existantes
- **Tous les enregistrements** conservés
- **Relations** maintenues

Statistiques

Élément	Quantité	Description
Tables	89	Toutes les tables converties
Clés étrangères	68	Relations entre tables
Tables avec données	36	Tables contenant des enregistrements
Enregistrements	400+	Données existantes préservées

Tables principales

Tables système

- `users` - Utilisateurs (4 enregistrements)
- `roles` - Rôles (3 enregistrements)
- `permissions` - Permissions (38 enregistrements)
- `migrations` - Historique des migrations

Tables métier (MaxiDoc)

- `agents` - Agents/employés
- `courriers` - Gestion des courriers
- `documents` - Gestion documentaire
- `taches` - Gestion des tâches
- `directions` - Structure organisationnelle
- `classeurs` - Organisation des documents

Tables de liaison

- `pivot_*` - Tables de relations many-to-many
- `model_has_*` - Relations Spatie/Permission

⚠ Points d'attention

Ordre d'exécution

1. **D'abord** : Migrations de tables (ordre automatique)
2. **Ensuite** : Migration des clés étrangères
3. **Enfin** : Seeder pour les données

Clés étrangères

- Toutes les relations sont configurées avec `onDelete('cascade')`
- Les contraintes sont désactivées temporairement pendant le seeding

Compatibilité

- **PostgreSQL 12+**
- **Laravel 9+**
- **PHP 8.0+**

Dépannage

Erreur de clé étrangère

```
# Réinitialiser complètement  
php artisan migrate:fresh --seed
```

Problème de permissions

```
# Vérifier les permissions Spatie  
php artisan permission:cache-reset
```

Erreur de seeding

```
# Exécuter le seeder individuellement  
php artisan db:seed --class=DatabaseSeeder
```

Modifications apportées

Par rapport au SQL original

1. **Noms d'index raccourcis** (respect limite 63 caractères)
2. **Types de données adaptés** à PostgreSQL
3. **Syntaxe Laravel** pour la portabilité
4. **Clés étrangères séparées** pour éviter les erreurs de dépendance
5. **Seeder optimisé** pour l'insertion en lot

Avantages des migrations Laravel

- **Versioning** de la base de données
- **Rollback** possible
- **Portabilité** entre environnements
- **Collaboration** d'équipe facilitée
- **Tests** automatisés possibles

Résultat final

Votre base de données MaxiDoc est maintenant :

- **Compatible PostgreSQL**
- **Optimisée Laravel**
- **Prête pour la production**
- **Sans erreurs de transaction**
- **Avec toutes les données**

Support

En cas de problème :

1. Vérifiez la configuration PostgreSQL
2. Assurez-vous que Laravel est à jour
3. Consultez les logs Laravel (`storage/logs/laravel.log`)
4. Vérifiez les permissions de base de données

Votre application MaxiDoc est prête à fonctionner avec PostgreSQL !